

Diseño e implementación de un monitor de recursos para Linux con Python, concurrencia y persistencia SQLite

Julio Cesar Blacio Machuca Ariel Jose Llumiquinga Ñacato
Carrera de Ingeniería de Software
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador

Resumen—La observación de recursos es una actividad esencial para comprender el estado de un sistema operativo, pero puede resultar abstracta cuando se estudia solo mediante comandos aislados. Este trabajo presenta el diseño y la implementación de una aplicación de terminal para Linux desarrollada en Python 3. La propuesta organiza seis módulos de monitoreo mediante el patrón Modelo–Vista–Controlador: CPU, memoria y *swap*, procesos, usuarios conectados, sistemas de archivos y red. Los datos se obtienen directamente del sistema de archivos virtual */proc* y, cuando corresponde, de comandos nativos ejecutados de forma controlada. Como parte de los requisitos de sistemas operativos, la aplicación crea explícitamente hilos con `threading.Thread` y un proceso hijo con `os.fork()`, cuya comunicación se realiza mediante una tubería. Las capturas completas se persisten en SQLite y se administran con operaciones CRUD. La verificación ejecutada en Ubuntu bajo WSL registró 56 pruebas correctas, sin fallos. El resultado muestra una base didáctica trazable que integra observación, concurrencia y persistencia sin sustituir las interfaces del sistema por dependencias de monitoreo de alto nivel.

Palabras clave—Linux, monitoreo de recursos, Python, concurrencia, SQLite, sistema de archivos *proc*

I. INTRODUCCIÓN

La administración de recursos permite relacionar la actividad de procesos, memoria, almacenamiento y comunicaciones con el comportamiento observable de un sistema operativo. En particular, los conceptos de procesos, planificación, memoria y entrada/salida requieren mecanismos concretos para pasar de la descripción teórica a una observación verificable [1]. Linux expone parte de ese estado mediante interfaces del núcleo y comandos del espacio de usuario, lo que permite construir herramientas acotadas y transparentes para propósitos formativos.

El proyecto *Linux Resource Monitor* responde a esa necesidad con una aplicación interactiva de consola, orientada a observar y registrar el estado de un equipo Linux sin modificar procesos, red, memoria ni sistemas de archivos. Su alcance integra la lectura de recursos, la demostración explícita de hilos y procesos, y la persistencia de capturas completas. No se plantea como sustituto de herramientas especializadas ni como medidor de rendimiento.

La contribución del trabajo es reunir en una sola aplicación de terminal el monitoreo de seis dominios, la lectura directa de */proc*, comandos nativos, el uso explícito de `threading.Thread` y `os.fork()`, y operaciones CRUD

sobre SQLite. La estructura también deja evidencia automatizada para que las afirmaciones funcionales puedan revisarse en Linux.

El objetivo técnico no se limita a enumerar valores: busca conservar la procedencia de cada dato desde la fuente del sistema hasta su presentación o almacenamiento. Para ello se definieron contratos estructurados entre capas, una política de fallos parciales para la consulta en vivo y una regla más estricta para persistir: una captura se guarda solamente cuando contiene los seis módulos obligatorios. Esta distinción permite observar un sistema cambiante sin confundir una vista incompleta con un registro histórico completo.

II. FUNDAMENTOS Y TRABAJOS RELACIONADOS

El sistema de archivos virtual */proc* publica información generada por el núcleo y por procesos en ejecución; por ello es una interfaz adecuada para consultar contadores y descriptores sin requerir una base de datos intermedia [2]. En este trabajo se emplea para CPU, memoria, carga y tráfico de red, mientras que las sesiones, procesos y puntos de montaje se obtienen con utilidades nativas que presentan esas vistas del sistema.

Las dos clases de fuente tienen propiedades distintas. Los archivos de */proc* representan el estado al momento de cada lectura, mientras que las utilidades producen una instantánea cuya composición puede cambiar antes de que se presente. Por esta razón, el diseño no exige que un conteo de procesos coincida después con otra ejecución de *ps*; exige registros bien formados y campos trazables. De igual manera, los contadores acumulados de red se conservan como bytes y paquetes, sin interpretarlos como tasas si no existe un intervalo de medición.

Desde la perspectiva de sistemas operativos, separar la obtención, presentación y coordinación evita que el formato de la terminal se mezcle con el acceso a recursos. Esta decisión facilita contrastar las mediciones, conservarlas como capturas y controlar los errores por módulo. Además, permite estudiar la diferencia entre hilos que comparten el proceso y un hijo creado por `fork()`, sin ocultar esos mecanismos mediante bibliotecas de monitoreo de alto nivel [1].

III. METODOLOGÍA

A. Desarrollo incremental

El desarrollo se organizó en cinco semanas. La primera definió requerimientos, arquitectura MVC y esquema relacional; la segunda incorporó CPU, memoria y lectura de `/proc`; la tercera agregó disco, red, usuarios y procesos; la cuarta integró hilos, `fork()` y CRUD; y la quinta se destinó a integración, pruebas, manuales y evidencias. Esta secuencia mantuvo cada avance asociado a una fuente de datos y a criterios de aceptación observables.

B. Flujo de una captura

Una solicitud de estado general atraviesa cuatro etapas. Primero, el controlador registra una función recolectora por módulo. Segundo, el coordinador crea un objeto `Thread` para cada función, inicia los seis y espera su terminación. Tercero, consolida tres estructuras separadas: datos, errores y evidencias; esta última conserva nombre de módulo, nombre de hilo e instantes UTC de inicio y fin. Finalmente, la vista puede presentar los datos disponibles y advertir los errores. Si el usuario decide registrar el estado, el controlador CRUD aplica una validación adicional antes de delegar la transacción al repositorio.

La separación entre consulta y persistencia es deliberada. En una consulta en vivo, el fallo de una fuente no elimina la información obtenida por las demás. En cambio, `consolidar_captura` verifica que existan CPU, memoria, discos, red, procesos y usuarios, y que el diccionario de errores esté vacío. Si falta un módulo, se genera un error controlado y no se abre una operación de escritura para una captura incompleta.

C. Fuentes y cálculos

La CPU se describe con `/proc/cpuinfo`, `/proc/loadavg` y dos lecturas de `/proc/stat`. La utilización no se infiere de la carga promedio: se calcula a partir de los deltas entre tiempo total e inactivo,

$$U_{CPU} = 100 \left(1 - \frac{\Delta T_{idle}}{\Delta T_{total}} \right), \quad (1)$$

donde ambos contadores proceden de la línea agregada de CPU. Para memoria, se conserva la distinción entre memoria libre y disponible: la memoria usada es `MemTotal - MemAvailable`, y el *swap* usado es `SwapTotal - SwapFree`. La red usa `/proc/net/dev` para contadores e `ip` para direcciones; `ps`, `who` y `df -P -B1` complementan los datos de procesos, usuarios y sistemas montados.

D. Contratos de datos

Los modelos no devuelven bloques de terminal. CPU y memoria producen diccionarios; disco, red, procesos y usuarios producen listas de diccionarios, un elemento por montaje, interfaz, proceso o sesión. Entre las claves estables se encuentran `porcentaje_uso`, `mem_disponible_mb`, `punto_montaje`, `usuario_propietario`, `direccion_ip` e `inicio_sesion`. El controlador

conserva estas estructuras y la vista es la responsable de agregar unidades, redondear a dos decimales y construir tablas ASCII.

Cada parser acepta texto y puede probarse sin consultar el equipo actual. La red combina los contadores de `/proc/net/dev` con una sola dirección por interfaz: prioriza la primera IPv4 y usa IPv6 como alternativa; si no existe una dirección, conserva `None`. El disco analiza bytes de `df -P -B1` antes de convertirlos para presentación o persistencia. En usuarios, `inicio_sesion` mantiene el instante informado por `who`; la duración se calcula al mostrarlo y no reemplaza el dato almacenado.

E. Concurrencia y seguridad

Se crean seis hilos independientes, uno por módulo, antes de esperar su finalización. Los resultados, errores y evidencias se protegen con un bloqueo para consolidar una respuesta parcial cuando una lectura falla. Solo después de finalizar esos hilos se inicia el proceso hijo; este orden evita invocar `fork()` con otros hilos activos. El hijo lee una métrica de carga, comunica un mensaje JSON por una tubería y el padre lo recolecta con `waitpid()`.

La demostración comprueba además que no existan otros hilos activos antes de crear el hijo. Padre e hijo cierran el extremo de la tubería que no utilizan; el hijo finaliza con `os._exit()` y el padre convierte el estado devuelto por `waitpid()` en un código de salida. Un código distinto de cero o una tubería vacía se transforma en un error comprensible. La base de datos no participa en ese flujo y ninguna conexión SQLite se comparte con hilos o con el proceso hijo.

F. Fallos y estados cambiantes

Los comandos se ejecutan con una lista de argumentos, salida y error capturados, modo texto, tiempo límite de cinco segundos, `LC_ALL=C` y sin `shell=True` [3]. Un retorno no nulo se convierte en `RuntimeError`; la ausencia de un comando, un tiempo agotado, un archivo de `/proc` inexistente o un contenido incompleto propagan una excepción que el hilo registra con el nombre del módulo. Así, el menú recibe mensajes controlados en lugar de una terminación abrupta.

Los parsers ignoran líneas vacías o no conformes cuando pueden continuar y rechazan campos obligatorios ausentes cuando el resultado perdería significado. Este criterio es relevante para tablas que cambian durante la observación: un proceso puede terminar entre dos consultas y una interfaz puede carecer de IP. La aplicación admite listas vacías y direcciones no disponibles, pero no inventa valores para completar la captura.

G. Estrategia de verificación

Las pruebas unitarias validan funciones de análisis contra archivos de muestra para que los resultados no dependan solo de la máquina actual. Las pruebas de persistencia usan bases SQLite temporales, incluyendo las operaciones CRUD y la eliminación en cascada. Las integraciones que requieren `fork()` se limitan explícitamente a Linux. La ejecución

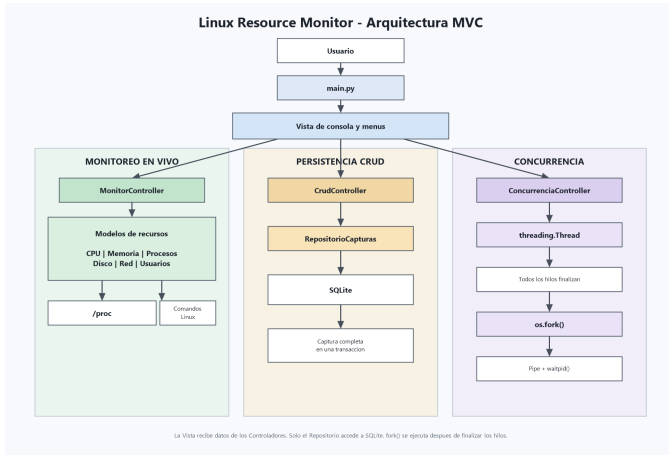


Fig. 1. Arquitectura MVC, recolección concurrente y persistencia del monitor.

documentada se realizó en Ubuntu bajo WSL; por tanto, este artículo no extiende esa evidencia a otros sistemas.

La verificación cubre también contratos entre capas: que el controlador preserve errores parciales, que una captura incompleta sea rechazada, que las conexiones activen claves foráneas, que la actualización afecte solo metadatos y que la eliminación retire las filas hijas. Para la interfaz se prueban opciones inválidas, identificadores no positivos, cancelación de borrado, mensajes de EXITO, ADVERTENCIA y ERROR, y el límite de listados extensos.

IV. ARQUITECTURA Y DESARROLLO DE LA SOLUCIÓN

A. Separación MVC

La Fig. 1 resume la organización. El modelo lee y transforma datos del sistema, y encapsula el repositorio SQLite. La vista solicita entradas por teclado y presenta menús, tablas y mensajes. El controlador coordina la recolección, las validaciones, la concurrencia y el flujo CRUD. Esta separación conserva contratos de datos estructurados entre capas y evita consultas SQL o lecturas de `/proc` en la vista.

La fachada `MonitorController` permite solicitar un módulo o el estado general sin exponer cómo se obtiene. `CrudController` recibe el resultado concurrente, exige la captura completa y traduce errores de SQLite a una excepción de aplicación. El repositorio es la única pieza que conoce sentencias SQL; de esta forma, la navegación de terminal y las reglas de integridad pueden probarse mediante dobles sin ejecutar comandos Linux.

B. Módulos de monitoreo

La Tabla I relaciona cada módulo con su fuente principal y el resultado que entrega al controlador. Las invocaciones de comandos se realizan con listas de argumentos, salida capturada, tiempo límite y sin `shell=True`; el uso de `subprocess` conserva un límite claro entre la aplicación y los comandos del sistema [3]. Las interfaces de Python usadas para procesos e hilos corresponden a las APIs documentadas de `os` y `threading` [4], [5].

TABLE I
TRAZABILIDAD ENTRE MÓDULO, FUENTE Y DATOS RECOLECTADOS.

Módulo	Fuente principal	Datos obtenidos
CPU	<code>/proc/cpuinfo</code> , <code>/proc/stat</code> , <code>/proc/loadavg</code>	Procesadores lógicos, modelo, frecuencia, carga y uso
Memoria	<code>/proc/meminfo</code>	Total, libre, disponible, usada y <i>swap</i>
Procesos	<code>ps</code>	PID, nombre, estado y propietario
Usuarios	<code>who</code>	Usuario, terminal e inicio de sesión
Disco	<code>df -P -B1</code>	Punto de montaje, bytes y porcentaje de uso
Red	<code>/proc/net/dev</code> , <code>ip</code>	Interfaz, dirección y contadores de tráfico

El módulo de CPU separa carga y utilización: las tres cargas provienen de `/proc/loadavg`, mientras que el porcentaje usa los deltas de `/proc/stat`. El conteo se denomina procesadores lógicos porque surge de las entradas `processor` de `/proc/cpuinfo`; no se presenta como núcleos físicos. En memoria se validan cinco campos obligatorios antes de convertir de KiB a MiB. Para procesos, usuarios y disco se analiza una ejecución de `ps`, `who` o `df`, respectivamente, lo que conserva una instantánea coherente con la fuente invocada.

C. Concurrencia y persistencia

La recolección inicia los seis objetos `Thread`, los une antes de consolidar el resultado y conserva errores por módulo. El proceso hijo se crea mediante `os.fork()`, envía sus datos por `pipe` y se recolecta sin dejar procesos huérfanos. Esta secuencia hace visible la diferencia entre coordinación de hilos y comunicación entre procesos.

Las capturas se almacenan en una tabla principal con métricas relacionadas de CPU, memoria, disco, procesos, red y usuarios. Cada conexión activa las claves foráneas y cada captura completa se escribe en una transacción, de modo que un fallo requerido revierte la operación. Las relaciones usan eliminación en cascada para retirar las métricas asociadas cuando se elimina una captura [6], [7]. El CRUD permite crear, listar y consultar capturas, actualizar sus metadatos y eliminarlas mediante el repositorio.

El diseño relacional distingue cardinalidades. CPU y memoria tienen una relación uno a uno con `capturas`: su clave primaria es también clave foránea. Disco, procesos, red y usuarios tienen relaciones uno a muchos y claves propias, porque una captura puede incluir varios elementos de cada tipo. La tabla padre conserva fecha, etiqueta, comentario y usuario de registro; la operación de actualización modifica solo etiqueta y comentario, nunca las métricas observadas.

La escritura abre una conexión independiente y entra en el contexto transaccional antes de insertar la fila padre y las seis familias de métricas. Si una clave requerida no existe, un tipo de contrato es incorrecto o SQLite rechaza una fila, el contexto revierte el conjunto y el bloque `finally` cierra la conexión. En la lectura, el repositorio reconstruye el mismo

TABLE II
RESULTADOS VERIFICADOS DE LA EJECUCIÓN DOCUMENTADA.

Validación	Resultado	Estado
Suite completa	56 pruebas	0 fallos
Estado general	6 módulos	0 errores
CRUD SQLite	Crear/listar/actualizar/eliminar	Correcto
Concurrencia	6 hilos + 1 hijo	Exit status 0

contrato empleado por la vista: un diccionario para CPU y memoria, y listas para las relaciones uno a muchos.

D. Criterios de interfaz terminal

El menú principal y el historial se operan con opciones numeradas. Una entrada inválida conserva el bucle activo; los identificadores deben ser enteros positivos y la eliminación exige escribir SI. Los listados de procesos y sistemas de archivos se limitan inicialmente a 20 filas e indican cuántos registros se muestran respecto del total. Estas decisiones evitan que una instantánea extensa desplace indefinidamente el contexto de la terminal.

La salida no depende del color. Los porcentajes se acompañan con NORMAL, ADVERTENCIA o CRITICO; los valores incluyen unidades y dos decimales, y el estado general muestra fecha de actualización. Las ausencias se expresan como *No disponible* o mediante mensajes específicos para listas vacías. Por ello, la interfaz sigue siendo comprensible en una terminal de texto simple y mediante teclado.

V. RESULTADOS Y VALIDACIÓN

La Tabla II resume una ejecución verificada el 12 de julio de 2026 en Ubuntu bajo WSL. La suite se ejecutó con `python3 -m unittest discover -s tests`; los demás registros aplicaron el controlador de monitoreo, un repositorio SQLite temporal y la demostración de concurrencia. Los valores son una instantánea de verificación funcional, no un punto de referencia de rendimiento ni una comparación de plataformas.

La evidencia de estado general registró los módulos CPU, memoria, discos, procesos, red y usuarios, con un diccionario de errores vacío. En la prueba de persistencia se creó una captura, se listó, se actualizaron sus metadatos y se eliminó; el listado final quedó sin registros. En la demostración de concurrencia se generaron seis evidencias con nombres `monitor-` y marcas de inicio y fin, seguidas de un hijo con PID distinto al padre y código de salida cero.

La suite combina pruebas deterministas y verificaciones de integración. Los parsers de CPU, memoria, disco, red, procesos y usuarios consumen archivos de muestra; el repositorio se crea en un directorio temporal y comprueba tanto el detalle reconstruido como la cascada; los controladores se ejercitan con recolectores y repositorios sustituibles; y los casos de Linux validan el flujo integrado y el `fork()` real. Esta composición reduce la dependencia de valores efímeros sin eliminar la comprobación final sobre el sistema objetivo.

```

Evidence: hilos_y_fork
Requirements: RF-07, RF-08, RF-09
Executed at (UTC): 2026-07-12T19:14:04.985173+00:00
Command: /usr/bin/python3 -c from controller.monitor_controller import MonitorController; r=MonitorController().get_evidence('hilos'); print({'hilos': r['recoleccion']['evidencias'], 'errores': r['recoleccion']['errores']})
Return code: 0

--- STDOUT ---
{'hilos': [{'modulo': 'memoria', 'hilo': 'monitor-memoria', 'inicio': '2026-07-12T19:14:04.872770+00:00', 'fin': '2026-07-12T19:14:04.873995+00:00'}, {'modulo': 'discos', 'hilo': 'monitor-discos', 'inicio': '2026-07-12T19:14:04.872770+00:00', 'fin': '2026-07-12T19:14:04.878737+00:00'}, {'modulo': 'procesos', 'hilo': 'monitor-procesos', 'inicio': '2026-07-12T19:14:04.872770+00:00', 'fin': '2026-07-12T19:14:04.879655+00:00'}, {'modulo': 'usuarios', 'hilo': 'monitor-usuarios', 'inicio': '2026-07-12T19:14:04.872770+00:00', 'fin': '2026-07-12T19:14:04.879751+00:00'}, {'modulo': 'red', 'hilo': 'monitor-red', 'inicio': '2026-07-12T19:14:04.872554+00:00', 'fin': '2026-07-12T19:14:04.879872+00:00'}, {'modulo': 'cpu', 'hilo': 'monitor-cpu', 'inicio': '2026-07-12T19:14:04.871699+00:00', 'fin': '2026-07-12T19:14:04.871699+00:00'}], 'errores': []}, {'parent_pid': 683, 'child_pid': 694, 'child_parent_pid': 683, 'exit_status': 0, 'carga_promedio_1m': 0.43}]
--- STDERR ---

```

Fig. 2. Evidencia de la ejecución de seis hilos y un proceso hijo en WSL.

Los criterios de aceptación se interpretaron según la naturaleza de cada dato. Se exige igualdad de claves y fórmulas para CPU y memoria, registros parseables para comandos, una dirección primaria o ausencia explícita en red, continuidad ante fallos parciales para la vista en vivo y atomicidad para el historial. No se exige igualdad exacta entre conteos tomados en instantes diferentes ni se generalizan los tiempos observados en una ejecución.

VI. CONCLUSIONES

El monitor integra los requisitos funcionales de observación de CPU, memoria, procesos, usuarios, disco y red en una aplicación de terminal para Linux. La separación MVC mantiene diferenciadas la lectura de recursos, la coordinación y la presentación; a su vez, el repositorio concentra la persistencia y las operaciones CRUD sobre SQLite.

La demostración de seis hilos y un hijo ilustra una decisión de orden relevante: los hilos terminan antes de iniciar `fork()`, y el padre recupera al hijo mediante una tubería y `waitpid()`. La transacción para cada captura completa y las claves foráneas con cascada sostienen la consistencia de los datos almacenados. La verificación documentada respalda el comportamiento descrito, sin atribuirle propiedades de rendimiento.

Como limitaciones, las lecturas dependen de las interfaces Linux disponibles y representan estados cambiantes del sistema. Trabajos futuros pueden estudiar la portabilidad de la capa de obtención o muestras de mayor duración, conservando la separación arquitectónica y sin afirmar que dichas extensiones formen parte de la versión actual.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. John Wiley & Sons, 2019.
- [2] The Linux Kernel Documentation, “The /proc filesystem,” <https://docs.kernel.org/filesystems/proc.html>, accessed: Jul. 17, 2026.
- [3] Python Software Foundation, “subprocess — subprocess management,” <https://docs.python.org/3/library/subprocess.html>, accessed: Jul. 17, 2026.
- [4] —, “os — miscellaneous operating system interfaces,” <https://docs.python.org/3/library/os.html>, accessed: Jul. 17, 2026.
- [5] —, “threading — thread-based parallelism,” <https://docs.python.org/3/library/threading.html>, accessed: Jul. 17, 2026.
- [6] SQLite Project, “Sqlite foreign key support,” <https://www.sqlite.org/foreignkeys.html>, accessed: Jul. 17, 2026.
- [7] —, “Transaction,” https://www.sqlite.org/lang_transaction.html, accessed: Jul. 17, 2026.